

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



BÁO CÁO KIẾN TRÚC MÁY TÍNH

Thiết kế và Mô phỏng Hệ thống an toàn của ô tô bằng ngôn ngữ Verilog

Nhóm 8:	Nguyễn Đăng Bình	20237305	Trưởng nhóm
	Phùng Huy Hoàng	20237336	Thành viên
	Phan Mạnh Hùng	202418908	Thành viên
	Nguyễn Lê Hải Long	20227059	Thành viên
	Nguyễn Tuấn Mạnh	20237360	Thành viên

GVHD: PGS. TS. Nguyễn Đình Hân

Hà Nội, 20 tháng 12 năm 2025

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



BÁO CÁO KIẾN TRÚC MÁY TÍNH

Thiết kế và Mô phỏng Hệ thống an toàn của ô tô bằng ngôn ngữ Verilog

Nhóm 8:	Nguyễn Đăng Bình	20237305	Trưởng nhóm
	Phùng Huy Hoàng	20237336	Thành viên
	Phan Mạnh Hùng	202418908	Thành viên
	Nguyễn Lê Hải Long	20227059	Thành viên
	Nguyễn Tuấn Mạnh	20237360	Thành viên

GVHD: PGS. TS. Nguyễn Đình Hân

Hà Nội, 20 tháng 12 năm 2025

Danh sách thành viên

STT	Họ và tên	MSSV	Công việc	Đóng góp
1	Nguyễn Lê Hải Long	20227059	Làm hệ thống cảm biến va chạm và túi khí, làm hệ thống phanh	25 %
2	Nguyễn Tuấn Mạnh	20237360	Làm hệ thống cảm biến cân nặng, sơ đồ mạch hệ thống tương tác kiểm thử trực tiếp	25 %
3	Nguyễn Đăng Bình	20237305	Làm hệ thống cảm biến va chạm và túi khí, viết báo cáo và làm slide	20 %
4	Phùng Huy Hoàng	20237336	Làm testbench và mô phỏng tình huống bằng Python	15 %
5	Phan Mạnh Hùng	202418908	Làm hệ thống phanh và testbench	15 %

MỤC LỤC

Lời mở đầu	1
1 GIỚI THIỆU	2
1.1 Tên đề tài	2
1.2 Mô tả dự án và Bài toán	2
1.3 Các yêu cầu của hệ thống	3
2 PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ HỆ THỐNG	4
2.1 Kiến trúc tổng quan hệ thống	4
2.2 Mô tả chi tiết các khối chức năng	5
2.2.1 Khối Đồng bộ hóa tín hiệu (Module Synchronizer)	5
2.2.2 Khối Phân loại hành khách (Module Occupant_Classifier)	5
2.2.3 Khối Điều khiển Túi khí (Module car_safety_system - ACU)	5
2.2.4 Khối An toàn chủ động (Module Active_Safety_Unit - ASU)	6
2.3 Công cụ và Môi trường thiết kế	6
3 HIỆN THỰC HÓA HỆ THỐNG	8
3.1 Module Tiện ích: Đồng bộ hóa tín hiệu (Synchronizer)	8
3.2 Module Cảm biến: Phân loại hành khách (Occupant Classifier)	9
3.3 Module Điều khiển: Hệ thống Túi khí (Airbag Control Unit)	12
3.4 Module Điều khiển: An toàn chủ động (Active Safety Unit)	15
3.5 Tích hợp hệ thống (System Top Level)	19
4 KẾT QUẢ MÔ PHỎNG VÀ KIỂM THỬ HỆ THỐNG	22
4.1 Môi trường và Kịch bản kiểm thử	22
4.2 Phân tích Giảm đồ xung	22
4.3 Sơ đồ cấu trúc phần cứng	24
4.4 Kiểm thử tương tác thời gian thực	25

4.5	Mô phỏng Trực quan cấp Hệ thống	26
4.5.1	Kịch bản 1: Phanh khẩn cấp tự động	26
4.5.2	Kịch bản 2: Hỗ trợ đánh lái tránh va chạm	27
4.5.3	Kịch bản 3: Kích hoạt túi khí trong va chạm không thể tránh khỏi	28
4.6	Kết luận chương	29
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	30
5.1	Tổng kết nội dung thực hiện	30
5.2	Đánh giá và Hạn chế	30
5.3	Hướng phát triển	31
5.4	Lời kết	31
	TÀI LIỆU THAM KHẢO	32
	PHỤ LỤC	33

Lời mở đầu

Kiến trúc máy tính là một trong những lĩnh vực cốt lõi của ngành công nghệ thông tin và kỹ thuật máy tính, đóng vai trò cầu nối quan trọng giữa phần mềm và phần cứng. Để vận dụng những kiến thức lý thuyết đã học vào thực tiễn, nhóm chúng em đã quyết định lựa chọn và thực hiện đề tài về thiết kế, mô phỏng hệ thống điều khiển túi khí trên ô tô sử dụng ngôn ngữ Verilog. Đây là một bài toán mang tính thực tế cao, đòi hỏi sự kết hợp chặt chẽ giữa tư duy logic và kỹ thuật thiết kế mạch số.

Trong quá trình thực hiện bài báo cáo này, chúng em đã nhận được sự hướng dẫn và truyền đạt kiến thức tận tình từ PGS. TS. Nguyễn Đình Hân. Chúng em xin gửi lời cảm ơn chân thành đến thầy vì đã giúp chúng em định hướng và tiếp cận với tư duy thiết kế phần cứng hiện đại.

Mặc dù đã nỗ lực hoàn thiện đề tài với tinh thần nghiêm túc nhất, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tế, bài báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để có thể hoàn thiện hơn kiến thức của mình trong tương lai.

Xin trân trọng cảm ơn.

Hà Nội, tháng 12 năm 2025

Nhóm 8

1 GIỚI THIỆU

1.1 Tên đề tài

Thiết kế và Mô phỏng Hệ thống An toàn ô tô sử dụng ngôn ngữ Verilog.

1.2 Mô tả dự án và Bài toán

Trong bối cảnh công nghệ ô tô hiện đại, hệ thống an toàn không còn hoạt động riêng lẻ mà đòi hỏi sự phối hợp chặt chẽ giữa An toàn thụ động nhằm giảm thiểu chấn thương khi va chạm và An toàn chủ động nhằm phòng tránh va chạm. Dự án này tập trung vào việc thiết kế, mô hình hóa và mô phỏng một bộ điều khiển trung tâm tích hợp cả hai khía cạnh trên dựa vào input là các mức chuyển giao thanh ghi.

Hệ thống được thiết kế để mô phỏng hoạt động của các thành phần phần cứng sau:

1. Hệ thống đầu vào:

- Cảm biến gia tốc: Đo lường lực va chạm theo trục dọc (X) và trục ngang (Y) để phát hiện va chạm trực diện hoặc bên hông.
- Cảm biến môi trường: Giả lập tín hiệu phát hiện vật cản (obstacle_in) và đo khoảng cách thực tế tới vật cản (distance_to_obstacle).
- Hệ thống cảm biến ghế ngồi: Mô phỏng mảng cảm biến áp suất (4 cảm biến/ghế) tại 4 vị trí ghế (Lái, Phụ, Sau trái, Sau phải) để xác định cân nặng và phân loại hành khách.
- Tín hiệu điều khiển từ tài xế: Bao gồm lực đạp phanh và mô-men xoắn đánh lái.

2. Khối xử lý trung tâm:

- Occupant Classifier: Module xử lý tín hiệu số DSP cơ bản, sử dụng thuật toán trung bình trượt để lọc nhiễu cảm biến ghế và phân loại đối tượng (Người lớn, Trẻ em, Ghế trống, Lỗi).
- Airbag Control Unit: "Bộ não" quản lý an toàn thụ động, sử dụng Máy trạng thái hữu hạn để quyết định thời điểm và vị trí kích nổ túi khí dựa trên mức độ nghiêm trọng của va chạm và vị trí người ngồi.
- Active Safety Unit: "Bộ não" quản lý an toàn chủ động, thực hiện tính toán vật lý gồm động năng và ma sát để kích hoạt Phanh khẩn cấp tự động và Trợ lực lái tránh né.

3. Hệ thống chấp hành:

- Các tín hiệu kích nổ túi khí (Driver, Passenger, Side airbags, Curtain airbags).
- Tín hiệu điều khiển lực phanh thủy lực và motor trợ lực lái điện.
- Hệ thống cảnh báo hình ảnh cho tài xế.

1.3 Các yêu cầu của hệ thống

Để đảm bảo tính thực tế và độ tin cậy của một hệ thống nhúng an toàn, thiết kế phải đáp ứng các yêu cầu kỹ thuật sau:

- Yêu cầu về Đồng bộ hóa tín hiệu:
 - Hệ thống phải xử lý được vấn đề bất đồng bộ do các cảm biến hoạt động độc lập với xung nhịp của vi xử lý.
 - Cần áp dụng kỹ thuật "Double Flopping", nghĩa là cần có 2 tầng xử lý dữ liệu, cho tất cả các đầu vào để ngăn chặn trạng thái bất ổn định, đảm bảo dữ liệu sạch trước khi đưa vào logic xử lý.
- Yêu cầu về Logic An toàn:
 - Cơ chế "Không nổ oan": Túi khí tuyệt đối không được bung tại các vị trí ghế trống để tiết kiệm chi phí sửa chữa, và không bung tại vị trí có trẻ em ngồi để tránh gây chấn thương ngược.
 - Cơ chế "Pre-crash Sensing": Hệ thống phải có khả năng hạ thấp ngưỡng kích hoạt túi khí khi Radar phát hiện sắp có va chạm (trạng thái WARNING), giúp phản ứng nhanh hơn so với trạng thái bình thường.
- Yêu cầu về Xử lý Động lực học:
 - Tính toán thời gian thực: Hệ thống phanh phải tính toán được lực phanh cần thiết dựa trên bình phương vận tốc (v^2) và khoảng cách tới vật cản.
 - Cơ chế ưu tiên: Hệ thống phải có khả năng chiếm quyền kiểm soát phanh từ tài xế nếu phát hiện nguy cơ va chạm cao mà tài xế không phản ứng đủ lực.
 - Cân bằng Phanh - Lái: Hệ thống phải tự động giảm lực phanh khi phát hiện tài xế đang đánh lái gấp để tránh hiện tượng khóa bánh, đảm bảo xe vẫn có khả năng điều hướng trong tình huống khẩn cấp.

2 PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ HỆ THỐNG

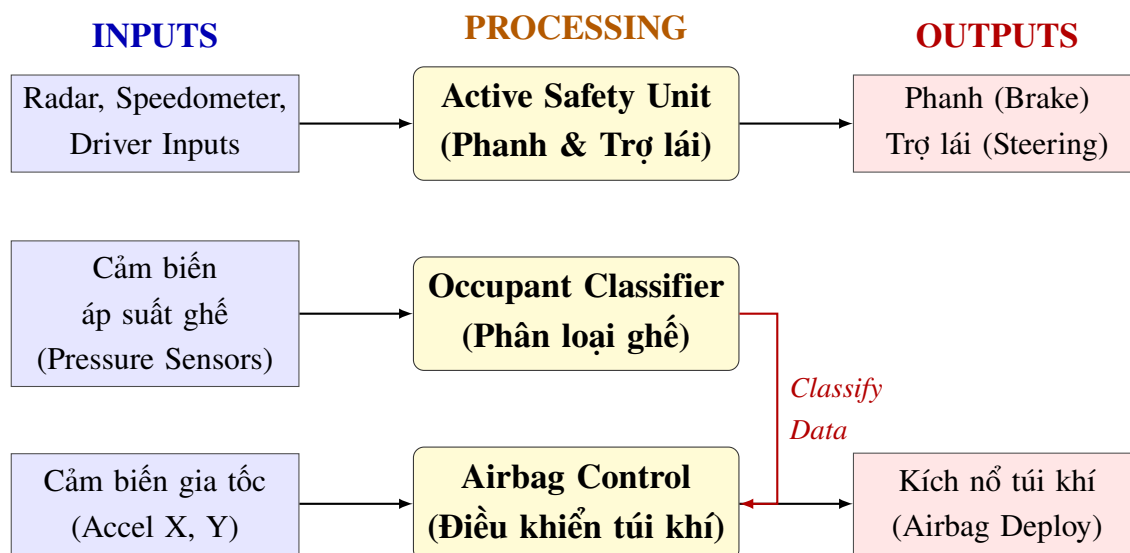
Phương pháp tiếp cận của đồ án là thiết kế hệ thống theo mô hình phân cấp từ trên xuống dưới (Top-down Design). Hệ thống được chia nhỏ thành các khối chức năng độc lập để đảm bảo tính tái sử dụng và dễ dàng kiểm lỗi. Ngôn ngữ mô tả phần cứng Verilog HDL được sử dụng để định nghĩa hành vi và cấu trúc của mạch dựa trên đầu vào là các mức chuyển giao thanh ghi (RTL).

2.1 Kiến trúc tổng quan hệ thống

Hệ thống được tích hợp trong module tổng quát `System_Top_Level`, đóng vai trò là bo mạch chủ kết nối các luồng dữ liệu. Kiến trúc bao gồm hai nhánh xử lý song song:

1. **Nhánh An toàn thụ động:** Chịu trách nhiệm thu thập dữ liệu từ 16 cảm biến áp suất ghế ngồi và cảm biến gia tốc để điều khiển hệ thống túi khí.
2. **Nhánh An toàn chủ động:** Chịu trách nhiệm thu thập dữ liệu vận tốc, khoảng cách và thao tác của tài xế để điều khiển hệ thống phanh và lái.

Toàn bộ hệ thống hoạt động đồng bộ theo sườn dương của xung nhịp (`clk`) và có cơ chế đặt lại bất đồng bộ (`rst_n`) để đưa hệ thống về trạng thái an toàn khi khởi động.



Ảnh 2.1 Sơ đồ đặc tả luồng dữ liệu của hệ thống (IPO Model)

2.2 Mô tả chi tiết các khối chức năng

2.2.1 Khối Đồng bộ hóa tín hiệu (Module Synchronizer)

Đây là lớp bảo vệ đầu tiên của hệ thống. Do các tín hiệu từ cảm biến bên ngoài (chân phanh, gia tốc,...) là tín hiệu không đồng bộ, nếu đưa trực tiếp vào mạch số sẽ gây ra hiện tượng bất ổn định (metastability).

- **Cấu trúc:** Sử dụng hai tầng D-Flip Flop mắc nối tiếp (kỹ thuật Double Flopping).
- **Hoạt động:** Tín hiệu ngẫu nhiên đi vào tầng 1 sẽ được chốt lại. Sau một chu kỳ xung nhịp, tín hiệu ổn định sẽ được chuyển sang tầng 2 và đưa vào hệ thống nội bộ.
- **Ứng dụng:** Module này được tái sử dụng tại đầu vào của tất cả các module con khác để đảm bảo độ tin cậy.

2.2.2 Khối Phân loại hành khách (Module Occupant_Classifier)

Module này thực hiện chức năng Xử lý tín hiệu số cơ bản để loại bỏ nhiễu và xác định đối tượng ngồi trên ghế.

- **Thuật toán lọc nhiễu:** Sử dụng bộ lọc trung bình trượt (Moving Average). Hệ thống lưu trữ 4 giá trị mẫu gần nhất vào một thanh ghi trượt (history), sau đó tính tổng và chia cho 4 (thực hiện bằng phép dịch bit phải $\gg 2$) để ra cân nặng trung bình, giúp làm mịn tín hiệu.
- **Logic phân loại:** So sánh cân nặng đã lọc với các ngưỡng được định nghĩa trước:
 - Dưới 10kg: Ghế trống (EMPTY).
 - Từ 10kg đến 30kg: Trẻ em (CHILD).
 - Trên 30kg: Người lớn (ADULT).
 - Tín hiệu bão hòa (Max value): Báo lỗi cảm biến (ERROR).

2.2.3 Khối Điều khiển Túi khí (Module car_safety_system - ACU)

Đây là trung tâm ra quyết định kích nổ, được thiết kế dựa trên mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM).

- **Các trạng thái hoạt động:**
 - IDLE: Trạng thái chờ, giám sát cảm biến gia tốc với ngưỡng va chạm cao (Hard Threshold).

- **WARNING:** Trạng thái cảnh báo khi Radar phát hiện vật cản. Tại đây, hệ thống hạ thấp ngưỡng kích hoạt (Soft Threshold) để tăng độ nhạy (cơ chế Pre-crash sensing).
- **FIRE:** Trạng thái kích nổ.
- **Logic an toàn (Interlock):** Tại trạng thái FIRE, hệ thống kiểm tra chéo với kết quả từ khối Phân loại hành khách. Tín hiệu kích nổ (deploy) chỉ được phát ra nếu vị trí đó được xác nhận là Người lớn (ADULT). Các trường hợp Trẻ em hoặc Ghế trống sẽ bị ngắt kích nổ để đảm bảo an toàn.

2.2.4 Khối An toàn chủ động (Module Active_Safety_Unit - ASU)

Module này thực hiện các tính toán vật lý thời gian thực để can thiệp vào hệ thống phanh và lái.

- **Mô hình vật lý:** Lực phanh cần thiết được tính toán dựa trên công thức động năng ($E \propto v^2$) và tỷ lệ nghịch với khoảng cách tới vật cản.
- **Cơ chế Trọng tải:** Hệ thống so sánh lực phanh do máy tính tính toán và lực phanh thực tế của tài xế. Nếu mức độ nguy hiểm vượt quá ngưỡng tối hạn (Critical Threshold) và tài xế không phản ứng, hệ thống sẽ tự động kích hoạt phanh tối đa.
- **Cơ chế Vòng tròn ma sát:** Khi kích hoạt chế độ khẩn cấp (enable_evasive), hệ thống giám sát mô-men xoắn vô lăng. Nếu phát hiện tài xế đánh lái gấp, hệ thống tự động giảm lực phanh tương ứng để tránh khóa bánh, giúp xe duy trì khả năng điều hướng.

2.3 Công cụ và Môi trường thiết kế

Quá trình hiện thực hóa, kiểm tra và mô phỏng hệ thống được thực hiện thông qua bộ công cụ mã nguồn mở phổ biến trong thiết kế vi mạch:

- **Ngôn ngữ thiết kế:** Verilog HDL (theo chuẩn IEEE 1364).
- **Trình biên dịch và mô phỏng (Icarus Verilog):** Đây là công cụ chính được sử dụng để biên dịch mã nguồn RTL và thực thi các bộ kịch bản kiểm thử (Testbench). Nó giúp kiểm tra tính đúng đắn về mặt logic của các thuật toán điều khiển phanh và máy trạng thái túi khí trong các tình huống giả lập.
- **Các công cụ hiển thị và trực quan hóa:**
 - *GTKWave:* Hiển thị kết quả mô phỏng (file .vcd) dưới dạng giản đồ xung, giúp xác minh thời điểm kích hoạt (timing) và quan sát chi tiết trạng thái các thanh ghi nội bộ.

- *DigitalJS*: Trực quan hóa sơ đồ nguyên lý trực tiếp từ mã nguồn Verilog và hỗ trợ mô phỏng tương tác thời gian thực, cho phép điều khiển thủ công các tín hiệu đầu vào để kiểm tra logic đầu ra.

3 HIỆN THỰC HÓA HỆ THỐNG

Phần này trình bày chi tiết mã nguồn Verilog RTL của các khối chức năng. Mã nguồn được tổ chức theo cấu trúc phân cấp, đi kèm với các chú thích chi tiết về giải thuật xử lý bên trong.

3.1 Module Tiện ích: Đồng bộ hóa tín hiệu (Synchronizer)

Module synchronizer là tầng bảo vệ đầu tiên, giúp loại bỏ hiện tượng bất ổn định (Metastability) khi tín hiệu đi từ miền không đồng bộ (cảm biến bên ngoài) vào miền đồng bộ của FPGA.

```
1 module synchronizer #(parameter WIDTH = 16) (  
2     input wire clk,  
3     input wire rst_n,  
4     input wire [WIDTH-1:0] async_in,    // Tín hiệu đầu vào  
        chưa đồng bộ  
5     output reg [WIDTH-1:0] sync_out    // Tín hiệu đầu ra đã  
        đồng bộ  
6 );  
7     // Sử dụng 2 tầng Flip-Flop nối tiếp để chống  
        Metastability  
8     reg [WIDTH-1:0] stage1;  
9  
10    // Khởi tạo giá trị ban đầu tránh trạng thái Unknown (X)  
11    initial begin  
12        stage1 = 0;  
13        sync_out = 0;  
14    end  
15  
16    always @(posedge clk or negedge rst_n) begin  
17        if (!rst_n) begin  
18            // Reset toàn bộ các tầng về 0  
19            stage1    <= 0;  
20            sync_out <= 0;  
21        end else begin  
22            // Tầng 1: Chấp nhận tín hiệu thô, có thể gặp  
                Metastability  
23            stage1    <= async_in;  
24            // Tầng 2: Lấy tín hiệu từ Tầng 1, đảm bảo đầu ra  
                ổn định theo xung nhịp hệ thống
```



```

25         sync_out <= stage1;
26     end
27 end
28 endmodule

```

Mã nguồn 1: Mã nguồn module Synchronizer (synchronizer.v)

Phân tích ý tưởng thiết kế:

- **Vấn đề:** Tín hiệu từ cảm biến thực tế (như áp suất, vận tốc) là tín hiệu không đồng bộ. Nếu đưa trực tiếp vào hệ thống số, hiện tượng *Metastability* (trạng thái logic lơ lửng giữa 0 và 1) có thể xảy ra, gây sai lệch toàn bộ logic phía sau.
- **Giải pháp:** Sử dụng kỹ thuật *Double-Flopping* (2 Flip-Flop nối tiếp). Flip-flop thứ nhất (stage1) sẽ hứng chịu sự bất ổn định, và flip-flop thứ hai (sync_out) sẽ đợi thêm một chu kỳ xung nhịp để trạng thái tín hiệu ổn định hẳn về mức logic 0 hoặc 1 trước khi đưa vào hệ thống xử lý chính.

3.2 Module Cảm biến: Phân loại hành khách (Occupant Classifier)

Module này chịu trách nhiệm xử lý dữ liệu thô từ 4 cảm biến áp suất, lọc nhiễu và đưa ra quyết định phân loại đối tượng ngồi trên ghế.

```

1 module Occupant_Classifier (
2     input wire clk,
3     input wire rst_n,
4
5     // Dấu vào thô từ 4 cảm biến áp suất (Giá trị 0-255)
6     input wire [7:0] SensorFrontLeft_raw,
7     input wire [7:0] SensorFrontRight_raw,
8     input wire [7:0] SensorBackLeft_raw,
9     input wire [7:0] SensorBackRight_raw,
10
11     output reg [7:0] weight_kg,    // Cân nặng đã được lọc
12                                     nhiễu
13     output reg [1:0] classify      // Kết quả phân loại:
14                                     EMPTY, CHILD, ADULT, ERROR
15 );
16
17 reg [9:0] current_total;
18 reg [9:0] history [0:3]; // Mang lưu lịch sử 4 mẫu gần
19                             nhất
20 wire [7:0] FL_sync, FR_sync, BL_sync, BR_sync;

```

```

19 // --- CAU HINH NGUONG PHAN LOAI ---
20 localparam [7:0] CHILD_LIMIT = 8'd10; // Duoi 10kg la
    Ghe trong
21 localparam [7:0] ADULT_LIMIT = 8'd30; // 10kg - 30kg
    la Tre em
22 localparam [7:0] MAX_KG = 8'd255; // Gia tri bao
    hoa cam bien
23 localparam [7:0] MAX_REAL_WEIGHT = 8'd200; // Gioi han
    can nang hop ly
24
25 // Ma hoa trang thai dau ra
26 localparam EMPTY = 2'b00;
27 localparam CHILD = 2'b01;
28 localparam ADULT = 2'b10;
29 localparam ERROR = 2'b11;
30
31 initial begin
32     weight_kg <= 0; classify <= EMPTY;
33     current_total <= 0;
34     history[0] <= 0; history[1] <= 0; history[2] <= 0;
        history[3] <= 0;
35 end
36
37 // Instance cac module dong bo hoa cho tung cam bien
38 synchronizer #(.WIDTH(8)) u_sync_1 (.clk(clk),
    .rst_n(rst_n), .async_in(SensorFrontLeft_raw),
    .sync_out(FL_sync));
39 synchronizer #(.WIDTH(8)) u_sync_2 (.clk(clk),
    .rst_n(rst_n), .async_in(SensorFrontRight_raw),
    .sync_out(FR_sync));
40 synchronizer #(.WIDTH(8)) u_sync_3 (.clk(clk),
    .rst_n(rst_n), .async_in(SensorBackLeft_raw),
    .sync_out(BL_sync));
41 synchronizer #(.WIDTH(8)) u_sync_4 (.clk(clk),
    .rst_n(rst_n), .async_in(SensorBackRight_raw),
    .sync_out(BR_sync));
42
43 always @(posedge clk or negedge rst_n) begin
44     if (!rst_n) begin
45         weight_kg <= 0; classify <= EMPTY; current_total
            <= 0;
46         history[0] <= 0; history[1] <= 0; history[2] <=

```

```

0; history[3] <= 0;
47 end else begin
48     // 1. Tổng hợp dữ liệu từ 4 cảm biến
49     current_total <= FL_sync + FR_sync + BL_sync +
        BR_sync;
50
51     // 2. Thực hiện thuật toán Cửa sổ trượt (Sliding
        Window)
52     history[0] <= current_total;
53     history[1] <= history[0];
54     history[2] <= history[1];
55     history[3] <= history[2];
56
57     // 3. Tính trung bình cộng: (S1+S2+S3+S4)/4
58     // Sử dụng phép dịch bit phải (>>2) thay cho phép
        chia để tối ưu tài nguyên
59     weight_kg <= (history[0] + history[1] +
        history[2] + history[3]) >> 2;
60
61     // 4. Logic so sánh và phân loại
62     // Kiểm tra lỗi cảm biến (Short circuit hoặc giá
        trị quá lớn)
63     if ((FL_sync == MAX_KG) || (FR_sync == MAX_KG) ||
64         (BL_sync == MAX_KG) || (BR_sync == MAX_KG) ||
        (weight_kg > MAX_REAL_WEIGHT)) begin
65         classify <= ERROR;
66     end
67     else if (weight_kg < CHILD_LIMIT)           classify
        <= EMPTY;
68     else if (weight_kg <= ADULT_LIMIT)           classify
        <= CHILD;
69     else                                           classify
        <= ADULT;
70     end
71 end
72 endmodule

```

Mã nguồn 2: Mã nguồn module Phân loại hành khách (Occupant_Classifier.v)

Phân tích giải thuật và Ý tưởng thiết kế:

- **Lọc nhiễu bằng Trung bình trượt (Moving Average Filter):** Cảm biến áp suất trên ô tô thường bị rung động (noise) khi xe di chuyển. Thay vì lấy giá trị tức thời,

module sử dụng mảng history để lưu 4 giá trị gần nhất. Giá trị đầu ra là trung bình cộng của 4 mẫu này. Việc này giúp làm mượt dữ liệu cân nặng.

- **Tối ưu hóa phần cứng:** Phép chia là tác vụ tốn tài nguyên trên FPGA. Do đó, kích thước của sổ lọc được chọn là 4 (lũy thừa của 2) để phép tính chia trung bình có thể thực hiện bằng phép dịch bit ($\gg 2$), giúp giảm độ trễ và tiết kiệm cổng logic.
- **Cơ chế phát hiện lỗi:** Nếu bất kỳ cảm biến nào báo giá trị tối đa (255) hoặc tổng trọng lượng vượt quá mức phi thực tế (200kg), hệ thống sẽ chuyển sang trạng thái ERROR để cảnh báo bảo trì, tránh kích hoạt túi khí sai.

3.3 Module Điều khiển: Hệ thống Túi khí (Airbag Control Unit)

Module này là trái tim của hệ thống an toàn thụ động, sử dụng Máy trạng thái hữu hạn (FSM) để ra quyết định bung túi khí dựa trên mức độ va chạm và loại hành khách.

```
1 module car_safety_system (  
2     input wire clk,  
3     input wire rst_n,  
4  
5     // INPUTS TU CAM BIEN GIA TOC & RADAR  
6     input wire signed [15:0] raw_accel_x,  
7     input wire signed [15:0] raw_accel_y,  
8     input wire raw_obstacle_det,  
9  
10    // Thông tin phân loại hành khách từ module Occupant  
11    // Classifier  
12    input wire [1:0] driver, pass_front, pass_back_left,  
13    pass_back_right,  
14  
15    // OUTPUTS KICH NO TUI KHI  
16    output reg deploy_driver, deploy_passenger,  
17    output reg deploy_front_left, deploy_front_right,  
18    output reg deploy_back_left, deploy_back_right,  
19    output reg deploy_rear_left, deploy_rear_right,  
20  
21    output wire [1:0] fsm_state  
22 );  
23  
24 // ... [Khai bao wire va sync nhu cu] ...  
25 // (Lưu ý: Code đóng bo hoa giu nguyen nhu phien ban  
26 // truoc)  
27  
28 // --- DINH NGHIA NGUONG VA CHAM ---
```

```

25     localparam signed [15:0] THRESHOLD_FRONT_HARD =
        -16'd5000; // Va cham manh truc dien
26     localparam signed [15:0] THRESHOLD_FRONT_SOFT =
        -16'd3000; // Va cham nhe
27     localparam signed [15:0] THRESHOLD_SIDE      =
        16'd3000; // Va cham hong xe
28
29     // --- DINH NGHIA TRANG THAI FSM ---
30     localparam IDLE = 2'b00;      // Binh thuong
31     localparam WARNING = 2'b01; // Phat hien vat can
        (Pre-crash)
32     localparam FIRED = 2'b10;     // Da kich no
33     localparam ADULT = 2'b10;     // Chi bung tui khi cho Nguoi
        lon
34
35     reg [1:0] current_state, next_state;
36
37     // --- LOGIC FSM (MAY TRANG THAI) ---
38     // 1. Chuyen trang thai
39     always @(posedge clk or negedge rst_n) begin
40         if (!rst_n) current_state <= IDLE;
41         else         current_state <= next_state;
42     end
43
44     // 2. Tinh toan trang thai ke tiep
45     always @(*) begin
46         next_state = current_state;
47         case (current_state)
48             IDLE: begin
49                 // Neu gia toc am dot ngot lon hon nguong
                    cung -> Va cham ngay lap tuc
50                 if (clean_accel_x <= THRESHOLD_FRONT_HARD)
                    next_state = FIRED;
51                 // Neu Radar phat hien vat can -> Chuyen sang
                    canh bao
52                 else if (clean_obstacle)
                    next_state = WARNING;
53             end
54             WARNING: begin
55                 // O che do canh bao, nguong kich no thap hon
                    (nhay hon)
56                 if (clean_accel_x <= THRESHOLD_FRONT_SOFT)

```

```

        next_state = FIRED;
        // Neu het vat can -> Ve trang thai nghi
    else if (!clean_obstacle)
        next_state = IDLE;
    end
    FIRED: next_state = FIRED; // Trang thai chot
        (Latch), khong quay lai
    default: next_state = IDLE;
endcase
end

// --- LOGIC DIEU KHIEN DAU RA (INTERLOCK) ---
// Chi kich no khi: Trang thai la FIRED (co va cham) VA
// Ghe co Nguai lon
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        deploy_driver <= 0; deploy_passenger <= 0;
        deploy_rear_left <= 0; deploy_rear_right <= 0;
    end else begin
        if (next_state == FIRED) begin
            if (driver == ADULT)                deploy_driver
                <= 1'b1;
            if (pass_front == ADULT)
                deploy_passenger <= 1'b1;
            if (pass_back_left == ADULT)
                deploy_rear_left <= 1'b1;
            if (pass_back_right == ADULT)
                deploy_rear_right <= 1'b1;
        end
    end
end

// --- LOGIC TUI KHI HONG (SIDE AIRBAGS) ---
// Xu ly doc lap voi FSM tui khi truoc
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        deploy_front_left <= 0; deploy_front_right <= 0;
        deploy_back_left <= 0; deploy_back_right <= 0;
    end else begin
        // Va cham ben Trai (Gia toc Y duong)
        if (clean_accel_y >= THRESHOLD_SIDE) begin
            if (driver == ADULT)

```

```

91         deploy_front_left <= 1'b1;
          if (pass_back_left == ADULT) deploy_back_left
            <= 1'b1;
92     end
93     // Va chạm bên Phải (Gia tốc Y âm)
94     if (clean_accel_y <= -THRESHOLD_SIDE) begin
95         if (pass_front == ADULT)
96             deploy_front_right <= 1'b1;
97             if (pass_back_right == ADULT)
98                 deploy_back_right <= 1'b1;
99     end
100 end
101 assign fsm_state = current_state;
102 endmodule

```

Mã nguồn 3: Mã nguồn module Điều khiển túi khí (car_safety_system.v)

Phân tích giải thuật và Ý tưởng thiết kế:

- **Máy trạng thái (FSM) 2 cấp độ:** Hệ thống không chỉ dựa vào gia tốc va chạm đơn thuần.
 - Trạng thái *IDLE*: Sử dụng ngưỡng kích hoạt cao (THRESHOLD_FRONT_HARD) để tránh kích hoạt nhầm khi xe đi qua ổ gà.
 - Trạng thái *WARNING*: Khi Radar phát hiện vật cản, hệ thống chuyển sang trạng thái "cảnh giác cao độ". Lúc này ngưỡng kích hoạt được hạ thấp (THRESHOLD_FRONT_SOFT), giúp túi khí bung nhanh hơn ngay cả khi va chạm mềm.
- **Cơ chế khóa an toàn (Interlock Logic):** Đây là tính năng quan trọng nhất để tiết kiệm chi phí sửa chữa và đảm bảo an toàn. Tín hiệu kích nổ (deploy) là phép toán AND giữa điều kiện va chạm (FIRED) và điều kiện ghế ngồi (ADULT). Túi khí sẽ **không** bung nếu ghế trống hoặc chỉ có trẻ em (tránh gây chấn thương cho trẻ do lực nổ túi khí).

3.4 Module Điều khiển: An toàn chủ động (Active Safety Unit)

Module này thực hiện các tính toán vật lý thời gian thực để hỗ trợ phanh khẩn cấp và đánh lái, bao gồm cơ chế "Trọng tài" để giải quyết xung đột giữa người lái và hệ thống.

```

1 module Active_Safety_Unit (
2     // ... [Inputs/Outputs giu nguyen] ...
3     input wire [7:0]    current_velocity,        // m/s
4     input wire [15:0] distance_to_obstacle,    // cm
5     input wire [7:0]    async_driver_brake_pedal,
6     input wire signed [7:0] async_driver_steering_torque,
7     output reg [7:0]    final_brake_command,
8     output reg signed [7:0] steer_assist_command,
9     output reg          warning_visual_active
10 );
11
12 // ... [Khai bao dong bo hoa giu nguyen] ...
13
14 // --- THONG SO VAT LY ---
15 localparam [7:0] BRAKE_GAIN = 8'd10;
16 localparam [7:0] CRITICAL_BRAKE_THRESHOLD = 8'd250; //
17     // Nguong phanh nguy hiem
18 localparam [7:0] EVASIVE_ASSIST_TRIGGER = 8'd150; //
19     // Nguong kích hoạt hỗ trợ lái
20
21 reg [31:0] velocity_squared;
22 reg [31:0] required_brake_energy;
23 reg [7:0]  calculated_physics_brake;
24 reg [7:0]  traction_limit_brake, abs_steer_torque;
25 reg signed [15:0] raw_assist_cal;
26 reg enable_evasive_assist;
27 reg [7:0] tentative_brake;
28
29 always @(posedge clk or negedge rst_n) begin
30     if (!rst_n) begin
31         // ... [Reset values] ...
32     end else begin
33         // 1. TINH TOAN LUC PHANH LY THUYET
34         // Công thức:  $E \sim v^2 / d$  (Động năng tỉ lệ thuận
35         // với bình phương vận tốc)
36         velocity_squared = current_velocity *
37             current_velocity;
38
39         if (distance_to_obstacle > 0) begin
40             required_brake_energy = (velocity_squared *
41                 BRAKE_GAIN) / distance_to_obstacle;
42         end
43     end
44 end

```



```

37         // Bao hoa gia tri o muc 255 (Max phanh)
38         calculated_physics_brake <=
           (required_brake_energy >= 255) ? 8'd255 :
           required_brake_energy[7:0];
39     end else begin
40         calculated_physics_brake <= 8'd255; // Khoang
           cach = 0 -> Phanh chet
41     end
42
43     // 2. DIEU KHIEN BAN KINH MA SAT (TRACTION CIRCLE)
44     // Neu dang danh lai gap (Steer > 0), phai giam
           luc phanh de tranh truot banh (Skidding)
45     abs_steer_torque <= (clean_steer_torque < 0) ?
           -clean_steer_torque : clean_steer_torque;
46
47     // Kich hoat che do ho tro tranh ne (Evasive
           Assist)
48     if ((calculated_physics_brake >
           EVASIVE_ASSIST_TRIGGER) || (clean_brake_pedal
           > EVASIVE_ASSIST_TRIGGER))
49         enable_evasive_assist <= 1'b1;
50     else
51         enable_evasive_assist <= 1'b0;
52
53     if (enable_evasive_assist) begin
54         // Giam gioi han phanh khi luc danh lai tang
           len
55         if ((abs_steer_torque * 2) > 255)
           traction_limit_brake <= 0;
56         else traction_limit_brake <= 8'd255 -
           (abs_steer_torque * 2);
57
58         // Tang cuong luc tro lai (nhan 3 lan momen
           dau vao)
59         raw_assist_cal = clean_steer_torque * 3;
60     end else begin
61         traction_limit_brake <= 8'd255; // Không danh
           lai -> Full phanh
62         raw_assist_cal = 0;
63     end
64
65     // Bao ve (Clamping) luc tro lai trong khoang -80

```

```

66         den 80
        if (raw_assist_cal > 80)
            steer_assist_command <= 80;
67     else if (raw_assist_cal < -80)
            steer_assist_command <= -80;
68     else
            steer_assist_command <= raw_assist_cal[7:0];
69
70     // 3. BỎ TRONG TAI (ARBITER LOGIC)
71     // So sánh lực phanh máy tính tính toán vs. lực
        phanh của tài xế
72     if ((calculated_physics_brake >=
        CRITICAL_BRAKE_THRESHOLD) &&
        (clean_brake_pedal <
        CRITICAL_BRAKE_THRESHOLD)) begin
73         // Tính hướng nguy hiểm mà tài xế không phanh
            du -> Máy tính chiếm quyền (Override)
74         tentative_brake = 8'd255;
75         warning_visual_active <= 1'b1;
76     end else begin
77         // Tính hướng bình thường -> Tôn trọng quyền
            lái của tài xế
78         tentative_brake = clean_brake_pedal;
79         warning_visual_active <= 1'b0;
80     end
81
82     // Áp dụng giới hạn ma sát cuối cùng
83     if (enable_evasive_assist && (tentative_brake >
        traction_limit_brake))
84         final_brake_command <= traction_limit_brake;
85     else
86         final_brake_command <= tentative_brake;
87     end
88 end
89 endmodule

```

Mã nguồn 4: Mã nguồn module An toàn chủ động (Active_Safety_Unit.v)

Phân tích giải thuật và Ý tưởng thiết kế:

- **Mô hình vật lý:** Hệ thống tính toán lực phanh dựa trên động năng ($E = \frac{1}{2}mv^2$). Do đó, biến `velocity_squared` được tính toán để đảm bảo lực phanh tăng theo cấp số nhân với vận tốc, phản ánh đúng thực tế vật lý rằng chạy càng nhanh thì càng cần

quãng đường phanh dài hơn.

- **Lý thuyết Vòng tròn ma sát:** Một lớp xe có giới hạn ma sát tổng hợp. Nếu dùng hết ma sát để phanh, xe sẽ không thể đánh lái (bị trượt thẳng).

- Code thực hiện logic: `traction_limit_brake = 255 - (Steer * k)`.
- Ý nghĩa: Khi phát hiện tài xế đang cố gắng đánh lái tránh chướng ngại vật (`enable_evasive_assist`), hệ thống sẽ tự động *giảm lực phanh* để dành ma sát cho việc bẻ lái, giúp xe không bị mất kiểm soát hướng.

- **Cơ chế Trọng tài:** Đây là logic quyết định ai nắm quyền kiểm soát:

- Nếu tính toán thấy va chạm sắp xảy ra (`calculated_physics_brake` cao) mà tài xế đạp phanh quá nhẹ hoặc không đạp -> Hệ thống tự động phanh gấp (Override).
- Trong các trường hợp khác, hệ thống đóng vai trò thụ động, truyền trực tiếp tín hiệu từ bàn đạp phanh của tài xế ra cơ cấu chấp hành.

3.5 Tích hợp hệ thống (System Top Level)

Module `System_Top_Level` đóng vai trò là "Bo mạch chủ", thực hiện kết nối (Wiring) tất cả các module con lại với nhau theo sơ đồ khối đã thiết kế.

```
1 module System_Top_Level (
2     // ... [Khai báo Port như cũ] ...
3 );
4
5     // Đây nơi bố (Interconnect wires) kết nối giữa Occupant
6     // Classifier và Airbag Unit
7     wire [1:0] class_driver, class_pass, class_rear_left,
8         class_rear_right;
9     wire [7:0] w_dr, w_ps, w_rl, w_rr;
10
11     // --- KHOI 1: INSTANTIATE 4 MODULE PHAN LOAI GHE ---
12     // Mọi ghe sử dụng 1 module riêng biệt chạy song song
13     Occupant_Classifier OCM_DRIVER      (.clk(clk),
14         .rst_n(rst_n), .SensorFrontLeft_raw(dr_s1), ...
15         .classify (class_driver));
16     Occupant_Classifier OCM_PASSENGER   (.clk(clk),
17         .rst_n(rst_n), .SensorFrontLeft_raw(ps_s1), ...
18         .classify (class_pass));
19     Occupant_Classifier OCM_REAR_LEFT   (.clk(clk),
20         .rst_n(rst_n), .SensorFrontLeft_raw(rl_s1), ...
21         .classify (class_rear_left));
```

```

14 Occupant_Classifier OCM_REAR_RIGHT (.clk(clk),
    .rst_n(rst_n), .SensorFrontLeft_raw(rr_s1), ...
    .classify (class_rear_right));
15
16 // --- KHOI 2: KHOI DIEU KHIEN TRUNG TAM (ACU) ---
17 // Nhan du lieu phan loai tu Khoi 1 de quyet dinh bung
    tui khi
18 car_safety_system ACU_MAIN_UNIT (
19     .clk(clk), .rst_n(rst_n),
20     .raw_accel_x(accel_x_in), .raw_accel_y(accel_y_in),
    .raw_obstacle_det(obstacle_in),
21     // Noi day tin hieu phan loai vao cong input cua ACU
    .driver(class_driver), .pass_front(class_pass),
22     .pass_back_left(class_rear_left),
    .pass_back_right(class_rear_right),
23     .deploy_driver(deploy_driver), ...
    .fsm_state(system_state)
24 );
25
26 // --- KHOI 3: AN TOAN CHU DONG (ASU) ---
27 // Hoat dong doc lap voi tui khi, xu ly phanh va lai
28 Active_Safety_Unit ASU_BRAKE_STEER (
29     .clk(clk), .rst_n(rst_n),
30     .current_velocity(current_velocity),
31     .distance_to_obstacle(distance_to_obstacle),
32     .async_driver_brake_pedal(driver_brake_pedal),
33     .async_driver_steering_torque(driver_steering_torque),
34     .final_brake_command(ctrl_brake_force),
35     .steer_assist_command(ctrl_steer_assist),
36     .warning_visual_active(visual_warning)
37 );
38
39 endmodule
40
41

```

Mã nguồn 5: Mã nguồn module Tích hợp hệ thống (System_Top_Level.v)

Phân tích kiến trúc hệ thống:

- **Xử lý song song:** Trong module Top Level, 4 module Occupant_Classifier, module ACU và module ASU được khởi tạo độc lập. Trong FPGA, điều này có nghĩa là chúng hoạt động đồng thời. Việc phân loại hành khách ở ghế sau không hề làm chậm quá trình tính toán phanh của hệ thống an toàn chủ động, đảm bảo phản hồi

thời gian thực.

- **Luồng dữ liệu:** Dữ liệu được quy hoạch đi một chiều rõ ràng: Cảm biến → Đồng bộ hóa → Xử lý Logic → Cơ cấu chấp hành.

4 KẾT QUẢ MÔ PHỎNG VÀ KIỂM THỬ HỆ THỐNG

4.1 Môi trường và Kịch bản kiểm thử

Để kiểm chứng độ chính xác của hệ thống tích hợp (System Top Level), nhóm đã thiết lập môi trường mô phỏng (Testbench) bao gồm đầy đủ các kích thích đầu vào mô tả một tình huống tai nạn giao thông điển hình. Mục tiêu là kiểm tra sự phối hợp giữa ba module chính: `Occupant_Classifier`, `Active_Safety_Unit` và `car_safety_system`.

Kịch bản kiểm thử được xây dựng theo trình tự thời gian gồm 4 giai đoạn: Vận hành bình thường → Tiền va chạm (Active Safety) → Va chạm vật lý → Quyết định bung túi khí. Chi tiết 6 trường hợp kiểm thử (Test Cases) điển hình được tóm tắt gọn trong Bảng 4.1.

Bảng 4.1 Tóm tắt các kịch bản kiểm thử

TT	Kịch bản	Kích thích (Input)	Kết quả (Output)
1	Cảnh báo	obstacle=1, cự ly xa	State: IDLE → WARNING
2	Phanh gấp	Vel cao, Dist ↓ 10m	Brake Force = 255 (Max); Vel → 0
3	Né tránh	Phanh Max + Đánh lái mạnh	Steer Assist = ON; Brake < Max
4	Va chạm (Adult)	Accel $X \approx -5000$; Ghế > 15kg	Bung túi khí Lái + Phụ
5	Va chạm (Child)	Accel $X \approx -5000$; Ghế < 15kg	Chỉ bung túi khí Lái (Pass = OFF)
6	Va chạm Hồng	Accel $Y \approx 3000$ (Bên trái)	Chỉ bung túi khí Rèm trái

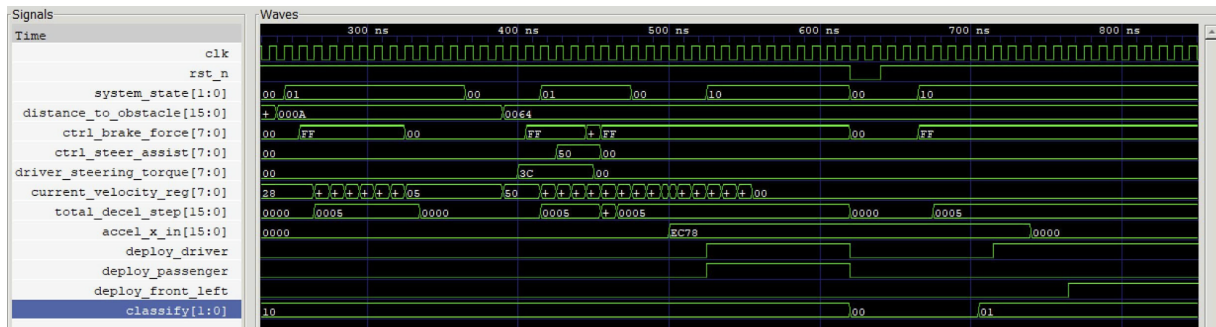
4.2 Phân tích Giải đồ xung

Kết quả mô phỏng tín hiệu thu được từ công cụ cho thấy sự tương tác logic chính xác giữa các tín hiệu vào và ra. Dưới đây là phân tích chi tiết dựa trên giải đồ xung thu được.

Quá trình mô phỏng tại Hình 4.2 được chia thành các mốc thời gian quan trọng như sau:

Trạng thái ban đầu ($t < 300ns$)

Tại thời điểm này, tín hiệu `system_state` đang ở mức 00 (IDLE). Tín hiệu phân loại ghế `classify` có giá trị 10, tương ứng với trạng thái *ADULT* (Người lớn). Điều này chứng tỏ module `Occupant_Classifier` đã xử lý xong dữ liệu từ các cảm biến áp



Ảnh 4.2 Giảm đồ xung mô phỏng toàn bộ quy trình từ phát hiện vật cản đến khi bung túi khí

suất (SensorFront...) và xác nhận ghế lái có người ngồi, đủ điều kiện để kích hoạt hệ thống bảo vệ.

Kích hoạt Phanh và Trợ lái khẩn cấp ($t \approx 300ns - 400ns$)

Khi xe phát hiện vật cản nguy hiểm:

- Tín hiệu khoảng cách distance_to_obstacle giảm xuống giá trị rất thấp (Hex: 000A).
- Hệ thống ngay lập tức phản hồi bằng cách đẩy tín hiệu điều khiển phanh ctrl_brake_force lên mức tối đa FF (tương đương 255). Điều này phù hợp với logic trong module Active_Safety_Unit, khi năng lượng phanh yêu cầu vượt quá ngưỡng tính toán.
- Tại thời điểm $t \approx 400ns$, tài xế thực hiện đánh lái (driver_steering_torque = 3C). Do hệ thống đang trong trạng thái tránh va chạm (biến nội bộ enable_evasive_assist lên mức 1), tín hiệu đầu ra trợ lực lái ctrl_steer_assist được kích hoạt với giá trị 50 nhằm hỗ trợ mô-men xoắn cho vô lăng.

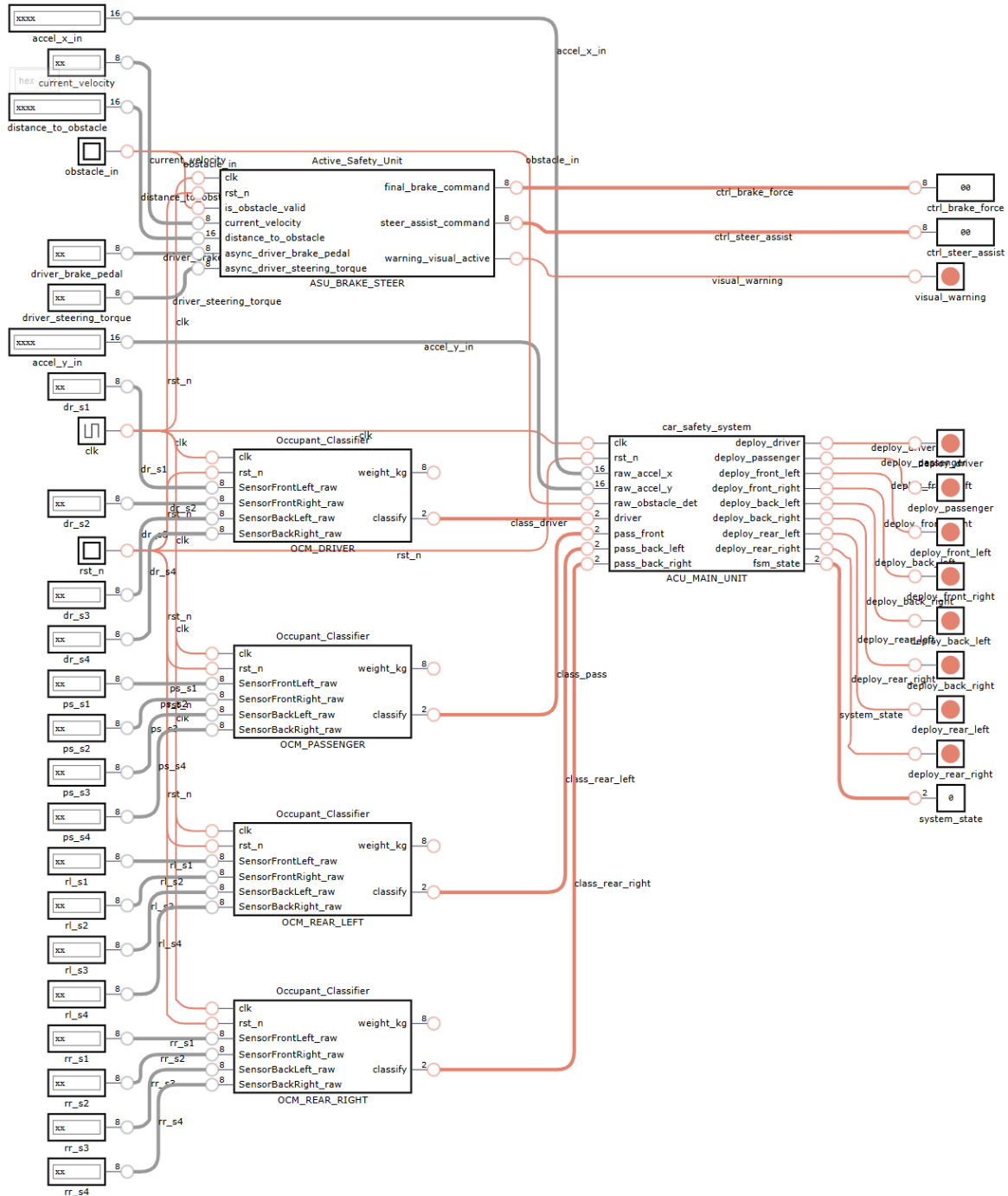
Xử lý Va chạm và Bung túi khí ($t > 550ns$)

Đây là giai đoạn quan trọng nhất của bài kiểm thử:

- Tín hiệu gia tốc accel_x_in thay đổi đột ngột về giá trị EC78. Trong hệ số bù 2 (16-bit), giá trị này tương ứng với số âm lớn (khoảng -5000), vượt qua ngưỡng THRESHOLD_FRONT_HARD (-5000) được định nghĩa trong mã nguồn.
- Máy trạng thái system_state chuyển từ 01 (WARNING) sang 10 (FIRED).
- Các tín hiệu deploy_driver và deploy_passenger chuyển từ mức logic 0 lên 1. Hệ thống quyết định bung túi khí vì hai điều kiện đồng thời thỏa mãn: va chạm mạnh (FIRED) và ghế có người ngồi (classify == ADULT).

4.3 Sơ đồ cấu trúc phần cứng

Trước khi tiến hành mô phỏng tín hiệu, mã nguồn Verilog được đưa qua quá trình tổng hợp trên công cụ DigitalJS để chuyển đổi từ ngôn ngữ mô tả phần cứng sang sơ đồ mạch logic mức cổng. Hình 4.3 dưới đây thể hiện kiến trúc tổng thể của module System_Top_Level.



Ảnh 4.3 Sơ đồ kết nối các module trên DigitalJS sau khi tổng hợp

Quan sát sơ đồ mạch, ta thấy hệ thống được tổ chức thành 3 phân hệ chính, tương

ứng với thiết kế trong mã nguồn:

- **Khối Phân loại hành khách (Occupant Classifier):** Gồm 4 module con nằm ở phía dưới bên trái sơ đồ. Các module này nhận dữ liệu thô từ cảm biến áp suất ghế (dr_s1 đến rr_s4), đi qua bộ lọc đồng bộ hóa, sau đó tính toán trọng lượng để xuất ra tín hiệu phân loại (classify) cho từng ghế.
- **Khối An toàn chủ động (Active Safety Unit):** Nằm ở phía trên sơ đồ, khối này xử lý các tín hiệu thời gian thực như vận tốc (current_velocity) và khoảng cách vật cản. Nó hoạt động độc lập với túi khí để đưa ra quyết định phanh và trợ lái tức thời.
- **Khối Điều khiển túi khí (Car Safety System):** Nằm ở phía bên phải, đóng vai trò là trung tâm xử lý va chạm. Khối này nhận tín hiệu gia tốc (accel_x) và các tín hiệu phân loại ghế từ khối Classifier để kích hoạt các ngõ ra túi khí (deploy_...) tương ứng.

Việc sơ đồ hiển thị đầy đủ các đường dây kết nối (wire) giữa các module chứng tỏ mã nguồn đã đảm bảo tính nhất quán về logic và sẵn sàng cho quá trình mô phỏng kiểm chứng.

4.4 Kiểm thử tương tác thời gian thực

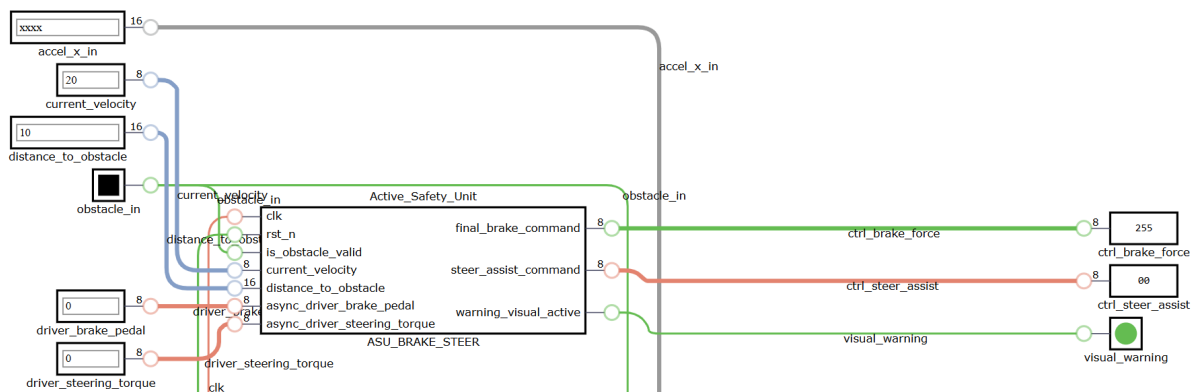
Bên cạnh việc mô phỏng tự động bằng Testbench, chúng tôi đã thực hiện kiểm thử thủ công trực tiếp trên sơ đồ RTL. Mục đích là xác minh tính năng "Phanh khẩn cấp tự động" trong tình huống tài xế mất tập trung.

Kịch bản kiểm thử: Xe đang di chuyển với tốc độ cao và gặp vật cản gần. Tài xế không thực hiện đạp phanh.

- **Thông số đầu vào:**
 - Vận tốc xe (current_velocity): 20 m/s (tương đương 72 km/h).
 - Khoảng cách vật cản (distance_to_obstacle): 10 m.
 - Phanh của tài xế (driver_brake_pedal): 0 (Không phản ứng).
- **Tính toán lý thuyết:** Theo logic trong module Active_Safety_Unit, năng lượng phanh cần thiết (E_{req}) được tính như sau:

$$E_{req} = \frac{v^2 \times Gain}{d} = \frac{20^2 \times 10}{10} = 400 \quad (4.1)$$

So sánh với ngưỡng nguy hiểm được lập trình là CRITICAL_THRESHOLD (250). Vì $400 > 250$, hệ thống được thiết kế để kích hoạt chế độ can thiệp tối đa.



Ảnh 4.4 Kết quả kiểm thử trên DigitalJS: Hệ thống tự động kích hoạt phanh tối đa (Output = 255) dù tài xế không thao tác.

Kết quả thực nghiệm:

Như thể hiện trong Hình 4.4, mặc dù tín hiệu đầu vào từ chân phanh (driver_brake_pedal) là 0, nhưng cổng ra ctrl_brake_force vẫn đạt giá trị cực đại 255. Đồng thời, đèn cảnh báo visual_warning bật sáng (màu xanh lá).

Điều này chứng minh thuật toán "Trọng tài" đã hoạt động chính xác: Hệ thống đã ghi đè quyền điều khiển của con người để ưu tiên an toàn khi phát hiện rủi ro va chạm nghiêm trọng.

4.5 Mô phỏng Trực quan cấp Hệ thống

Để kiểm chứng tính thực tế của mã nguồn Verilog trong môi trường vật lý giả lập, nhóm thực hiện đã xây dựng một bộ công cụ kiểm thử trực quan sử dụng thư viện cocotb kết hợp với pygame.

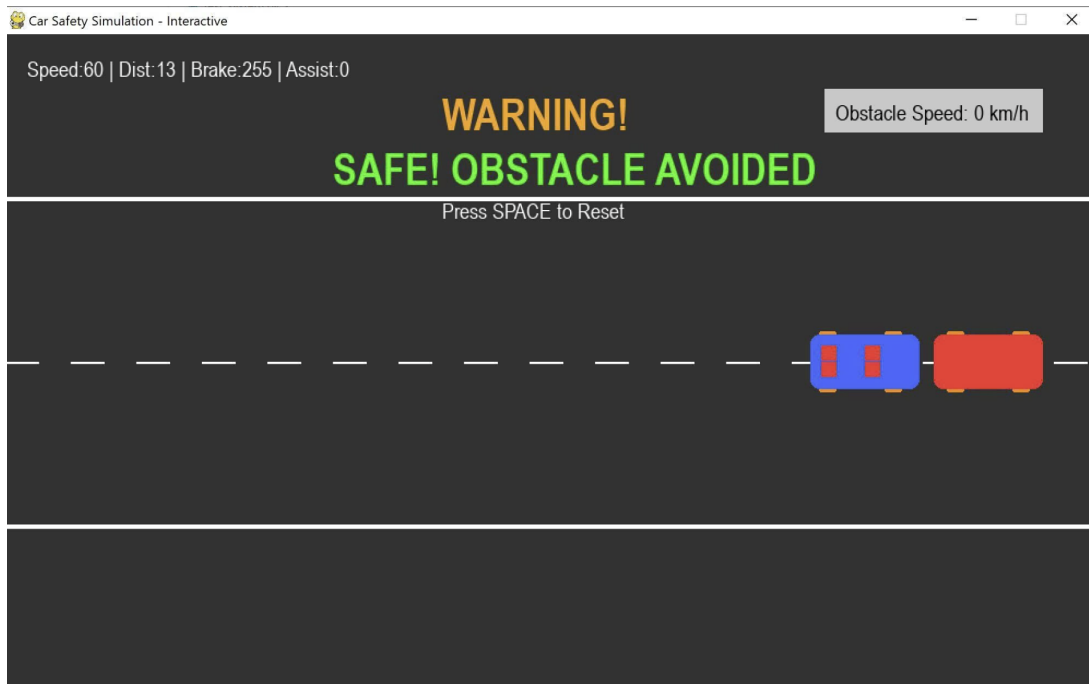
Phương pháp này cho phép nhúng trực tiếp nhân Verilog vào một vòng lặp mô phỏng thời gian thực, nơi các tín hiệu vật lý như vận tốc, ma sát và lực quán tính được tính toán bởi Python và phản hồi lại cho FPGA xử lý. Dưới đây là kết quả của 3 kịch bản vận hành điển hình.

4.5.1 Kịch bản 1: Phanh khẩn cấp tự động

Trong kịch bản này, xe di chuyển với vận tốc 60 km/h và gặp vật cản tĩnh (xe đỗ hỏng máy giữa đường). Tài xế mất tập trung và không thực hiện thao tác phanh.

Phân tích hoạt động:

- Cảm biến đo khoảng cách phát hiện nguy hiểm ($Distance < Safety_Limit$).
- Module Active_Safety_Unit tính toán năng lượng phanh cần thiết lớn hơn ngưỡng



Ảnh 4.5 Hệ thống tự động kích hoạt phanh cứng, xe dừng an toàn trước vật cản

CRITICAL.

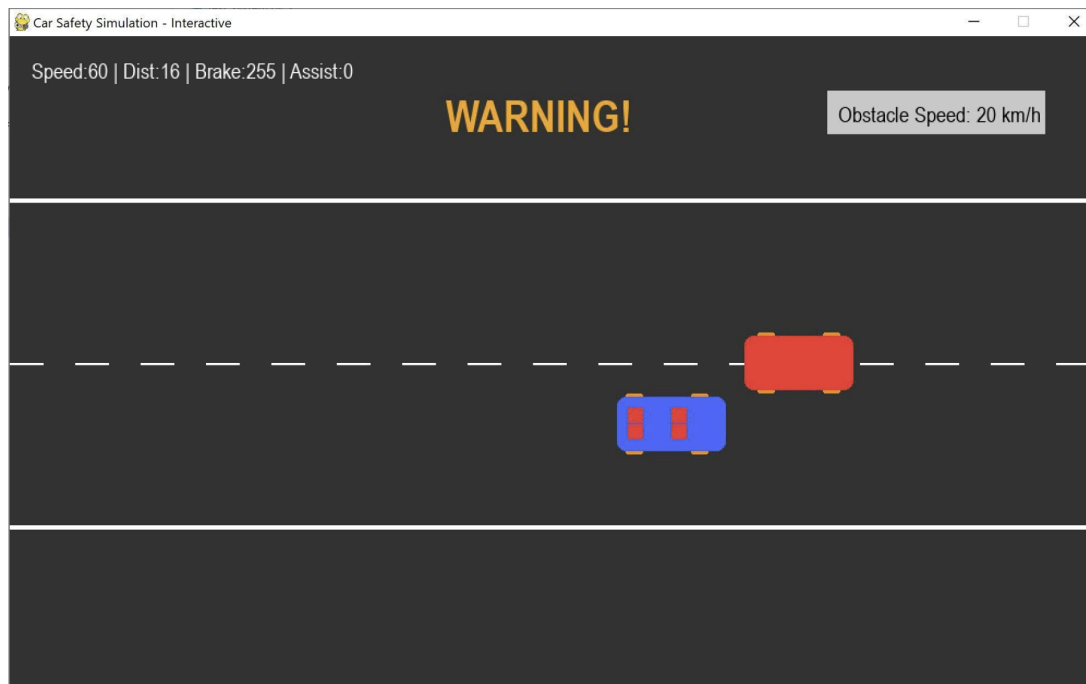
- Hệ thống kích hoạt tín hiệu `ctrl_brake_force` lên mức 255 (Max).
- **Kết quả:** Xe giảm tốc độ nhanh chóng và dừng lại hoàn toàn trước khi va chạm, dù tài xế không can thiệp. Màn hình hiển thị trạng thái "SAFE! OBSTACLE AVOIDED".

4.5.2 Kịch bản 2: Hỗ trợ đánh lái tránh va chạm

Kịch bản mô phỏng tình huống vật cản di chuyển chậm phía trước. Tài xế phát hiện và thực hiện đánh lái gấp để né tránh.

Phân tích hoạt động:

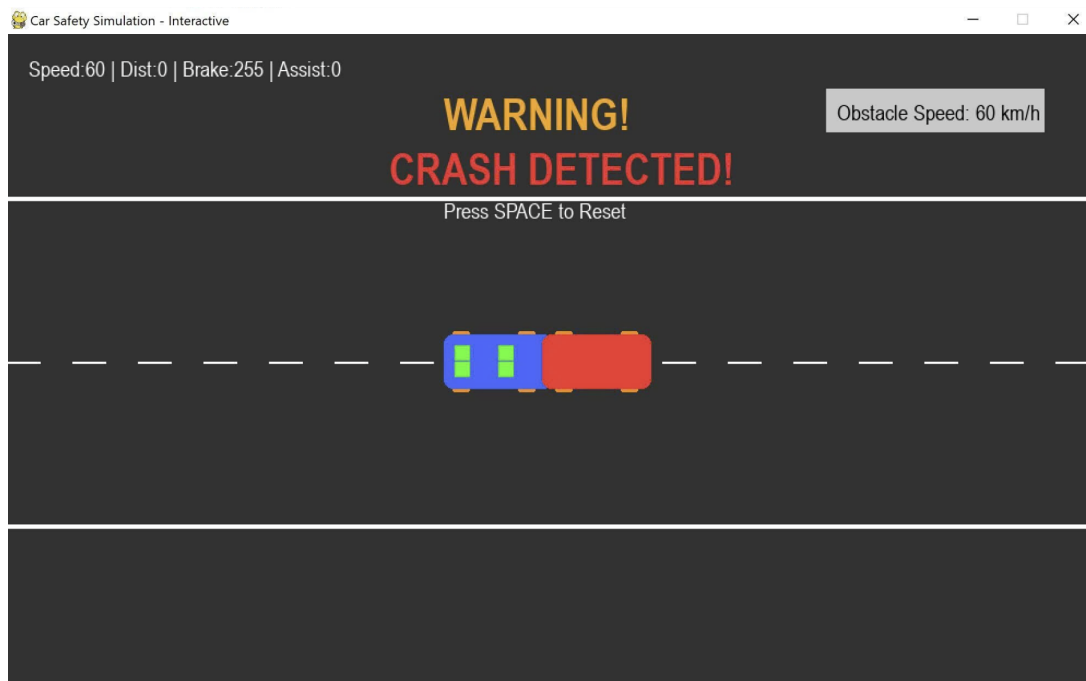
- Khi tài xế đánh lái (Steer Torque tăng cao), module `Active_Safety_Unit` áp dụng thuật toán *Vòng tròn ma sát*.
- Hệ thống tự động nhả bớt lực phanh (giảm từ 255 xuống thấp hơn) để dành độ bám đường cho bánh xe thực hiện việc chuyển hướng.
- Đồng thời, tín hiệu `ctrl_steer_assist` được kích hoạt để hỗ trợ thêm mô-men xoắn xoay vô lăng.
- **Kết quả:** Xe thực hiện cú đánh lái "mở neo" thành công, lách qua vật cản mà không bị trượt hay mất lái.



Ảnh 4.6 Cơ chế "Phanh mủ neo" giúp xe đánh lái mượt mà khi đang phanh gấp

4.5.3 Kịch bản 3: Kích hoạt túi khí trong va chạm không thể tránh khỏi

Trong tình huống vật cản lao đến với tốc độ cao, khoảng cách quá gần khiến hệ thống phanh và đánh lái không còn hiệu quả.



Ảnh 4.7 Hệ thống kích hoạt túi khí ngay lập tức khi phát hiện va chạm mạnh

Phân tích hoạt động:

- Va chạm xảy ra, gia tốc kế ghi nhận giá trị giảm tốc đột ngột (Input `accel_x` $\approx$ -6000, vượt ngưỡng Hard Threshold).
- Module `Occupant_Classifier` xác nhận có người ngồi ở các vị trí ghế.
- Module `car_safety_system` chuyển trạng thái FSM sang FIRED.
- **Kết quả:** Các tín hiệu `deploy_driver`, `deploy_passenger`... được kích hoạt đồng thời. Màn hình mô phỏng hiển thị cảnh báo "CRASH DETECTED" và minh họa túi khí bung (các ô vuông nhỏ trên ghế đổi màu), bảo vệ tính mạng hành khách.

4.6 Kết luận chương

Thông qua quá trình kiểm thử toàn diện, kết hợp giữa mô phỏng tự động và kiểm thử tương tác thủ công, nhóm thực hiện đi đến các kết luận sau về hoạt động của hệ thống:

- *Về chức năng:* Hệ thống đã đáp ứng đầy đủ các yêu cầu thiết kế đặt ra. Các khối chức năng hoạt động chính xác theo kịch bản: từ việc nhận diện vật cản, tính toán lực phanh/trợ lái theo công thức vật lý, đến quy trình quyết định bung túi khí dựa trên phân loại hành khách.
- *Về logic điều khiển:* Máy trạng thái hữu hạn (FSM) chuyển đổi ổn định giữa các trạng thái (IDLE $\rightarrow$ WARNING $\rightarrow$ FIRED) mà không xảy ra hiện tượng kẹt trạng thái hay bất ổn định.
- *Về khả năng tổng hợp:* Việc hệ thống chạy thành công trên DigitalJS và hiển thị sơ đồ RTL mức cổng chứng minh mã nguồn Verilog đảm bảo tính tổng hợp, cấu trúc mạch rõ ràng và các bộ đồng bộ hóa hoạt động hiệu quả.

Các kết quả mô phỏng khả quan này là cơ sở vững chắc khẳng định tính đúng đắn của thiết kế, tạo tiền đề để đánh giá hiệu năng tổng thể và định hướng phát triển trong chương tiếp theo.

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Tổng kết nội dung thực hiện

Báo cáo này đã trình bày quy trình thiết kế, hiện thực hóa và kiểm thử một hệ thống Vi mạch tích hợp (SoC) cho bài toán an toàn xe hơi. Hệ thống bao gồm ba phân hệ chính hoạt động song song: Phân loại hành khách (Occupant Classifier), Phanh/Trợ lái chủ động (Active Safety) và Điều khiển túi khí (Airbag Control).

Qua quá trình thực hiện bài tập lớn này, nhóm đã đạt được các kết quả cụ thể sau:

- **Về kiến trúc hệ thống:** Đã xây dựng thành công sơ đồ khối với sự tách biệt rõ ràng giữa các chức năng. Việc tích hợp các bộ đồng bộ hóa (Synchronizer) 2 tầng tại các ngõ vào đã giúp hệ thống xử lý tốt các tín hiệu bất đồng bộ từ môi trường mô phỏng.
- **Về giải thuật điều khiển:**
 - Hệ thống túi khí đã hiện thực hóa thành công máy trạng thái hữu hạn (FSM) đảm bảo quy trình kích nổ tuân thủ điều kiện an toàn "kép": Chỉ bung khi va chạm mạnh VÀ có người ngồi.
 - Hệ thống phanh khẩn cấp (AEB) đã áp dụng công thức vật lý động lực học để tính toán lực phanh và tự động chiếm quyền điều khiển khi phát hiện nguy hiểm.
- **Về kiểm thử:** Kết quả mô phỏng trên Testbench và kiểm thử tương tác (Interactive Testing) trên công cụ DigitalJS cho thấy hệ thống hoạt động ổn định, các tín hiệu ngõ ra phản hồi chính xác theo kịch bản tai nạn giả định.

5.2 Đánh giá và Hạn chế

Bên cạnh các kết quả đạt được, hệ thống mô phỏng hiện tại vẫn còn tồn tại một số hạn chế do giới hạn về thời gian và phạm vi của môn học:

- **Mô hình vật lý:** Các công thức tính toán khoảng cách và lực phanh đang dựa trên mô hình lý tưởng, chưa tính đến các yếu tố thực tế như ma sát mặt đường, độ trễ cơ học của phanh hay khối lượng tải trọng động của xe.
- **Giao thức kết nối:** Hiện tại các module đang giao tiếp với nhau qua các đường dây nối trực tiếp (Point-to-point). Trong thực tế công nghiệp ô tô, việc này thường được thực hiện qua mạng CAN Bus hoặc FlexRay để tối ưu hóa dây dẫn.

5.3 Hướng phát triển

Để hoàn thiện bài tập và tiệm cận hơn với các ứng dụng thực tế, nhóm đề xuất các hướng cải tiến sau:

1. **Nâng cấp bộ lọc tín hiệu:** Bổ sung bộ lọc Kalman hoặc bộ lọc trung bình trượt (Moving Average) vào đầu vào cảm biến gia tốc để xử lý nhiễu tốt hơn thay vì chỉ dùng đồng bộ hóa logic cơ bản.
2. **Triển khai bộ điều khiển PID:** Thay thế cơ chế phanh ON/OFF hiện tại bằng bộ điều khiển PID để lực phanh thay đổi mượt mà, giúp xe dừng lại êm ái hơn và tránh hiện tượng bó cứng.
3. **Thiết kế giao diện Bus:** Nghiên cứu tích hợp giao thức APB hoặc CAN đơn giản để các module trao đổi dữ liệu theo chuẩn công nghiệp, thay vì nối dây thủ công.

5.4 Lời kết

Bài tập lớn này đã giúp nhóm củng cố kiến thức về ngôn ngữ mô tả phần cứng Verilog, tư duy thiết kế mạch số và quy trình kiểm thử hệ thống nhúng. Đây là nền tảng quan trọng để tiếp cận các bài toán phức tạp hơn trong lĩnh vực thiết kế vi mạch và hệ thống nhúng trên ô tô.

TÀI LIỆU THAM KHẢO

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2020.
- [2] W. B. Ribbens, *Understanding Automotive Electronics: An Engineering Perspective*, 8th ed. Oxford, UK: Butterworth-Heinemann, 2017.
- [3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall Professional, 2003.

Phụ lục: Kế hoạch thực hiện dự án

Bảng dưới đây tóm tắt lộ trình làm việc và phân công nhiệm vụ chi tiết:

Tuần	Nội dung và Phân công công việc
1	Khởi tạo và Thiết kế hệ thống - Cả nhóm: Họp bàn ý tưởng, xác định tính năng cốt lõi và phạm vi đề tài. - Thiết kế sơ đồ khối (Block Diagram) và đặc tả module. - Phân chia công việc theo thể mạnh thành viên.
2	Triển khai Coding các Module cơ sở - Bình: Code Verilog cảm biến và chạm và máy trạng thái túi khí (FSM). - Mạnh: Thiết kế module xử lý cảm biến cân nặng (Occupant Classifier). - Long: Xây dựng khung hệ thống (System Architecture) và module đồng bộ hóa.
3	Mở rộng tính năng và Đồng bộ hóa - Cả nhóm: Rà soát logic, chuẩn hóa code module tuần 2. - Hùng & Long: Phát triển hệ thống phanh khẩn cấp và trợ lực lái (ASU). - Mạnh: Hướng dẫn trực quan hóa mạch logic trên DigitalJS.
4	Tích hợp, Kiểm thử và Báo cáo - Cả nhóm: Tích hợp module con vào module tổng (System Top Level). - Hùng & Hoàng: Chạy Testbench, xuất dạng sóng và mô phỏng. - Bình: Viết báo cáo tổng kết và thiết kế Slide thuyết trình.

Bảng 5.1 Tiến độ và phân công nhiệm vụ chi tiết